

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

1 Introduction

Recurrent neural networks, long short-term memory [12] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [29, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [31, 21, 13].

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Łukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

注意力就是你所需要的一切

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
husz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†] 多伦多大学
aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

摘要

主流的序列转换模型基于复杂的循环或卷积神经网络，这些网络包含一个编码器和一个解码器。性能最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，彻底摒弃了循环和卷积。在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时更具并行性，且训练所需时间显著减少。我们的模型在 WMT 2014 英语到德语翻译任务上达到了 28.4 BLEU，超过现有最佳结果（包括集成模型）2 BLEU 以上。在 WMT 2014 英语到法语翻译任务上，我们的模型在八块 GPU 上训练 3.5 天后，建立了新的单模型最先进水平 BLEU 分数 41.0，而训练成本仅为文献中最佳模型的一小部分。

1 引言

循环神经网络，尤其是长短期记忆 [12] 和门控循环 [7] 神经网络，已被牢固确立为序列建模和转换问题（如语言建模和机器翻译）中最先进的方法 [29, 2, 5]。此后，大量研究工作不断推动循环语言模型和编码器-解码器架构的边界 [31, 21, 13]。

*同等贡献。排名顺序随机。Jakob 提出用自注意力替代循环神经网络，并启动了评估这一想法的努力。Ashish 与 Illia 设计并实现了最初的 Transformer 模型，并在本项工作的各个方面发挥了关键作用。Noam 提出了缩放点积注意力、多头注意力以及无参数位置表示，并成为几乎参与每个细节的另一位成员。Niki 在最初的代码库和 tensor2tensor 中设计、实现、调优并评估了无数模型变体。Llion 也试验了新颖的模型变体，负责我们最初的代码库以及高效推理和可视化。Łukasz 和 Aidan 花费了无数漫长的日子设计和实现 tensor2tensor 的各个部分，替换了我们早先的代码库，大幅改进了结果并极大地加速了我们的研究。

[†]在 Google Brain 工作期间完成的工作。

[‡]在 Google Research 工作期间完成的工作。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t , as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [18] and conditional computation [26], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 16]. In all but a few cases [22], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [20], ByteNet [15] and ConvS2S [8], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [11]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 22, 23, 19].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [28].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [14, 15] and [8].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 29]. Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \dots, z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive [9], consuming the previously generated symbols as additional input when generating the next.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-

循环模型通常沿输入和输出序列的符号位置进行因子化计算。将位置与计算时间中的步长对齐，它们生成一系列隐藏状态 h_t ，作为前一隐藏状态 h_{t-1} 和位置 t 输入的函数。这种固有的顺序性质阻碍了训练样本内的并行化，而随着序列长度的增加，这一点变得尤为关键，因为内存限制限制了跨样本的批处理。近期工作通过分解技巧 [18]和条件计算 [26], 在计算效率方面取得了显著改进，同时在后者的情况下也提升了模型性能。然而，顺序计算的根本约束依然存在。

注意力机制已成为各种任务中引人注目的序列建模和转换模型不可或缺的组成部分，使得能够在不考虑输入或输出序列 [2, 16]中距离的情况下对依赖关系进行建模。然而，在大多数情况下 [22],，这种注意力机制是与循环网络结合使用的。

在这项工作中，我们提出了Transformer，一种摒弃循环的模型架构，转而完全依赖注意力机制来提取输入和输出之间的全局依赖关系。Transformer允许显著更多的并行化，并且在八个P100 GPU上训练仅十二小时后，就能达到翻译质量的最先进水平。

2 背景

减少顺序计算的目标也构成了扩展神经GPU[20]、ByteNet [15]和ConvS2S [8], 的基础，它们都使用卷积神经网络作为基本构建块，为所有输入和输出位置并行计算隐藏表示。在这些模型中，关联来自两个任意输入或输出位置的信号所需的操作数随位置之间的距离增长，对于ConvS2S是线性增长，对于ByteNet是对数增长。这使得学习远距离位置之间的依赖关系变得更加困难 [11]。在Transformer中，这被减少为常数数量的操作，尽管代价是由于对注意力加权位置进行平均而导致有效分辨率降低，我们通过第3.2节中描述的多头注意力来抵消这一影响。

自注意力，有时称为内部注意力，是一种将单个序列的不同位置相关联以计算该序列表示的注意力机制。自注意力已成功应用于多种任务，包括阅读理解、抽象摘要、文本蕴含以及学习与任务无关的句子表示 [4, 22, 23, 19]。

端到端记忆网络基于循环注意力机制而非序列对齐的循环，并已被证明在简单语言问答和语言建模任务上表现良好 [28]。

然而，据我们所知，Transformer 是第一个完全依赖自注意力来计算其输入和输出表示而无需使用序列对齐的循环神经网络或卷积的转换模型。在以下各节中，我们将描述Transformer，阐述自注意力的动机，并讨论其相对于 [14, 15] 和 [8]等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转换模型都具有编码器-解码器结构 [5, 2, 29]。在这里，编码器将输入符号表示序列 $(x_1, \dots, x_n)$ 映射为连续表示序列 $\mathbf{z} = (z_1, \dots, z_n)$ 。给定 $\mathbf{z}$ ，解码器随后一次一个元素地生成符号输出序列 $(y_1, \dots, y_m)$ 。在每一步中，模型都是自回归的 [9],，在生成下一个符号时将先前生成的符号作为额外输入。Transformer遵循这一整体架构，对编码器和解码器均使用堆叠自注意力和逐位置全连接层，分别如图1的左半部分和右半部分所示。

3.1 编码器和解码器堆叠

编码器: 编码器由 $N = 6$ 个相同层堆叠而成。每一层包含两个子层。第一个是多头自注意力机制，第二个是一个简单的、位置

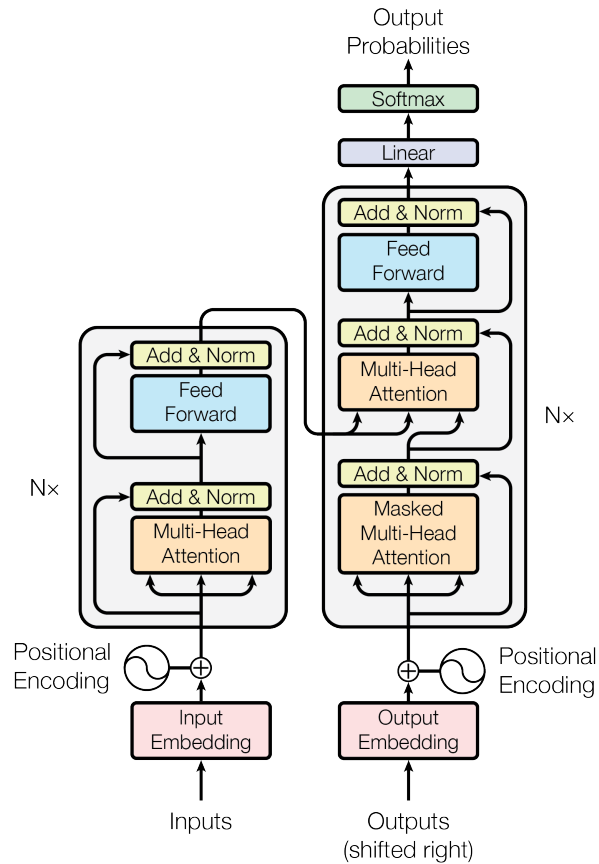


Figure 1: The Transformer - model architecture.

wise fully connected feed-forward network. We employ a residual connection [10] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the

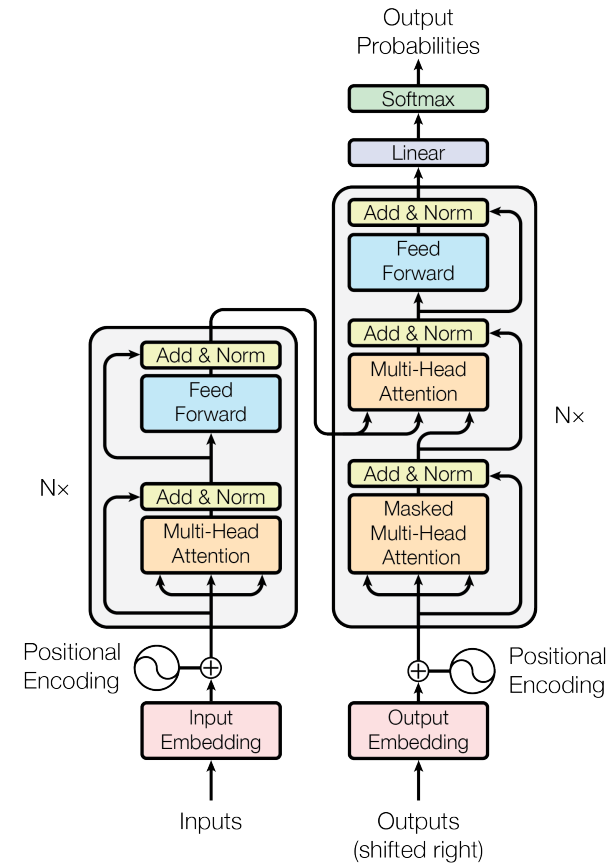


图1: Transformer - 模型架构。

感知的全连接前馈网络。我们在两个子层的每一个周围采用残差连接 [10]，随后进行层归一化 [1]。也就是说，每个子层的输出是 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是由子层本身实现的函数。为了便于这些残差连接，模型中的所有子层以及嵌入层产生的输出维度均为 $d_{\text{model}} = 512$ 。

解码器：解码器同样由 $N = 6$ 个相同层的堆叠组成。除了每个编码器层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层周围采用残差连接，随后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩码，结合输出嵌入偏移一个位置的事实，确保了对位置 i 的预测只能依赖于小于 i 的已知输出。

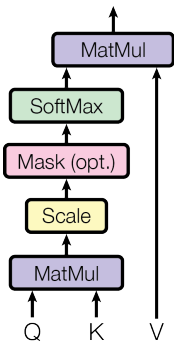
3.2 注意力

注意力函数可以描述为将一个查询和一组键值对映射到一个输出，其中查询、键、值和输出都是向量。输出被计算为值的加权和，其中分配给每个值的权重由查询与相应键的兼容性函数计算得出。

3.2.1 缩放点积注意力

我们将这种特定的注意力称为“缩放点积注意力”（图2）。输入由维度为 d_k 的查询和键，以及维度为 d_v 的值组成。我们计算

Scaled Dot-Product Attention



Multi-Head Attention

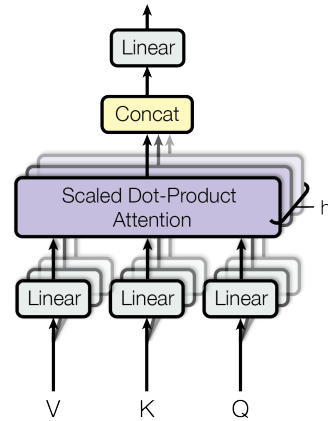


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

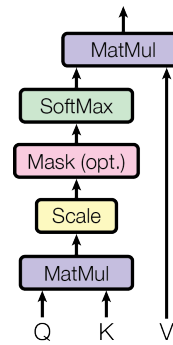
3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .

缩放点积注意力



多头注意力

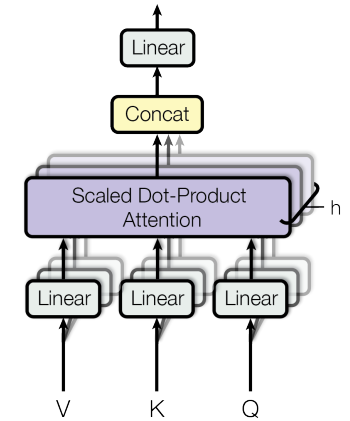


图2：（左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$ ，并应用softmax函数以获得值上的权重。

在实践中，我们同时对一组查询计算注意力函数，将它们打包成一个矩阵 Q 。键和值也被分别打包成矩阵 K 和 V 。我们计算输出矩阵如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

最常用的两种注意力函数是加性注意力 [2] 和点积（乘性）注意力。点积注意力与我们的算法相同，只是缩放因子为 $\frac{1}{\sqrt{d_k}}$ 。加性注意力使用具有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂度上相似，但在实践中点积注意力要快得多且更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值，两种机制表现相似，但对于较大的 d_k [3] 值，加性注意力在没有缩放的情况下优于点积注意力。我们推测对于较大的 d_k 值，点积的幅度会变大，将softmax函数推向具有极小梯度的区域⁴。为了抵消这一效应，我们将点积缩放 $\frac{1}{\sqrt{d_k}}$ 。

3.2.2 多头注意力

与其使用 d_{model} 维的键、值和查询执行单一的注意力函数，我们发现将查询、键和值分别用不同的、学习到的线性投影线性投影 h 次到 d_k 、 d_k 和 d_v 维是有益的。在这些投影后的查询、键和值版本上，我们随后并行地执行注意力函数，产生 d_v 维的输出值。这些值被拼接起来并再次投影，得到最终值，如图2所示。

多头注意力允许模型同时关注来自不同表示子空间和不同位置的信息。而使用单个注意力头时，求平均会抑制这一点。

⁴为了说明为什么点积会变大，假设 q 和 k 的分量是均值为0、方差为1的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为0，方差为 d_k 。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [31, 2, 8].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [24]. In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the

$$\text{多头}(Q, K, V) = \text{拼接}(\text{head}_1, \dots, \text{head}_h)W^O \text{其中}$$

$$\text{head}_i = \text{注意力}(QW_i^Q, KW_i^K, VW_i^V)$$

其中投影为参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

在本工作中，我们采用 $h = 8$ 个并行的注意力层，或称头。对于每个头，我们使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与具有完整维度的单头注意力相似。

3.2.3 注意力在我们模型中的应用

Transformer以三种不同方式使用多头注意力：

- 在“编码器-解码器注意力”层中，**查询来自前一个解码器层**，而**记忆键和值来自编码器的输出**。这使得解码器中的每个位置都能关注输入序列中的所有位置。这模仿了序列到序列模型（如[31, 2, 8]）中典型的编码器-解码器注意力机制。
- 编码器包含**自注意力层**。在自注意力层中，所有键、值和查询都来自同一位置，在此情况下即**编码器中前一层的输出**。编码器中的每个位置都能关注编码器前一层的所有位置。
- 同样，解码器中的自注意力层允许解码器中的每个位置关注解码器中直到并包括该位置的所有位置。我们需要防止解码器中的信息向左流动，以保留自回归性质。我们通过缩放点积注意力内部将softmax输入中对应于非法连接的所有值掩蔽掉（设为 $-\infty$ ）来实现这一点。参见图2。

3.3 逐位置前馈网络

除了注意力子层之外，我们的编码器和解码器中的每一层都包含一个全连接前馈网络，该网络被分别且相同地应用于每个位置。这由两个线性变换组成，中间夹有一个ReLU激活函数。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然线性变换在不同位置上是相同的，**但它们在层与层之间使用不同的参数**。另一种描述方式是将其视为两个卷积核大小为1的卷积。输入和输出的维度为 $d_{\text{model}} = 512$ ，而内层的维度为 $d_{ff} = 2048$ 。

3.4 嵌入与Softmax

与其他序列转换模型类似，我们使用学习嵌入将输入词元和输出词元转换为维度为 d_{model} 的向量。我们还使用通常的学习线性变换和softmax函数将解码器输出转换为预测的下一个词元概率。在我们的模型中，我们在两个嵌入层和pre-softmax线性变换之间共享相同的权重矩阵，类似于 [24]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$

3.5 位置编码

由于我们的模型不包含循环，也不包含卷积，为了使模型能够利用序列的顺序，我们必须注入一些关于序列中词元的相对或绝对位置的信息。为此，我们在输入嵌入中加入“位置编码”，

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [8].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

We also experimented with using learned positional embeddings [8] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, \dots, x_n)$ to another sequence of equal length $(z_1, \dots, z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [11]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [31] and byte-pair [25] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in

表1: 不同层类型的最大路径长度、每层复杂度和最少顺序操作数。 n 为序列长度, d 为表示维度, k 为卷积的卷积核大小, r 为受限自注意力中邻域的大小。

层类型	每层复杂度	顺序操作	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
自注意力 (受限)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

加在编码器和解码器堆叠的底部。位置编码与嵌入具有相同的维度 d_{model} , 因此两者可以相加。位置编码有许多选择, 包括可学习的和固定的 [8]。

在这项工作中, 我们使用不同频率的正弦和余弦函数:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

其中 pos 是位置, i 是维度。也就是说, 位置编码的每个维度对应一个正弦波。波长构成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这个函数是因为我们假设它能让模型轻松学会按相对位置进行注意力计算, 因为对于任何固定的偏移 k , PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们还尝试了改用学习的位置嵌入 [8], 并发现这两种版本产生的结果几乎相同 (见表3第 (E)行)。我们选择正弦版本, 是因为它可能使模型能够外推到比训练期间遇到的序列更长的序列长度。

4 为什么是自注意力

在本节中, 我们将自注意力层的各个方面与通常用于将一个变长符号表示序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ 的循环层和卷积层进行比较, 其中 $x_i, z_i \in \mathbb{R}^d$, 例如典型序列转换编码器或解码器中的隐藏层。为了说明我们使用自注意力的动机, 我们考虑三个期望目标。

其一是**每层的总计算复杂度**。其二是**可以并行化的计算量, 以所需的最少顺序操作数来衡量**。

其三是**网络中长距离依赖之间的路径长度**。学习**长距离依赖**是许多序列转换任务中的关键挑战。影响学习此类依赖能力的一个关键因素是**前向和后向信号在网络中必须经过的路径长度**。输入和输出序列中任意位置组合之间的这些**路径越短**, 学习长距离依赖就越容易 [11]。因此, 我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表1所述, 自注意力层以恒定数量的顺序执行操作连接所有位置, 而循环层则需要 $O(n)$ 次顺序操作。就计算复杂度而言, 当序列长度 n 小于表示维度 d 时, 自注意力层比循环层更快, 而这在最先机器翻译模型所使用的句子表示中最为常见, 例如词片[31] 和字节对 [25] 表示。为提升涉及极长序列任务的计算性能, 可将自注意力限制为仅考虑大小为 r 的邻域, 该邻域位于

the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(\log_k(n))$ in the case of dilated convolutions [15], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k . Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [31]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [17] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

5.4 Regularization

We employ three types of regularization during training:

Residual Dropout We apply dropout [27] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

输入序列中围绕各自输出位置的部分。这将把最大路径长度增加至 $O(n/r)$ 。我们计划在未来的工作中进一步研究这一方法。

一个卷积核宽度为 $k < n$ 的单层卷积层并不能连接输入和输出位置的所有对。要做到这一点，在连续卷积核的情况下，需要堆叠 $O(n/k)$ 个卷积层，或者在膨胀卷积的情况下需要 $O(\log_k(n))$ ，[15]，增加了网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，高出 k 倍。然而，可分离卷积 [6]，将复杂度大幅降低至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。然而，即使使用 $k = n$ ，可分离卷积的复杂度也相当于自注意力层与逐位置前馈层的组合，这正是我们模型中采用的方法。

作为附带好处，**自注意力可以产生更具可解释性的模型**。我们检查了模型中的注意力分布，并在附录中展示和讨论了相关示例。不仅各个注意力头清晰地学会了执行不同的任务，许多注意力头还表现出与句子的句法和语义结构相关的行为。

5 训练

本节描述了我们模型的训练方案。

5.1 训练数据与批处理

我们在标准的 WMT 2014 英德数据集上进行训练，该数据集包含约 450 万对句子。句子使用字节对编码 [3]，进行编码，其共享的源语言-目标语言词汇表包含约 37000 个词元。对于英法翻译，我们使用了明显更大的 WMT 2014 英法数据集，该数据集包含 3600 万对句子，并将词元拆分为 32000 个词片词汇表 [31]。句子对按近似序列长度进行批处理。每个训练批次包含一组句子对，其中大约有 25000 个源词元和 25000 个目标词元。

5.2 硬件与调度

我们在一台配备 8 个 NVIDIA P100 GPU 的机器上训练我们的模型。对于使用本文所述超参数的基础模型，每个训练步骤大约耗时 0.4 秒。我们将基础模型总共训练了 100,000 步，即 12 小时。对于大型模型（见表3底行所述），每步耗时为 1.0 秒。大型模型训练了 300,000 步（3.5 天）。

5.3 优化器

我们使用了 **Adam 优化器** [17]，参数为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 和 $\epsilon = 10^{-9}$ 。我们在训练过程中根据以下公式调整学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这相当于在前 $warmup_steps$ 个训练步骤中线性增加学习率，此后按步数的平方根倒数成比例地降低学习率。我们使用了 $warmup_steps = 4000$ 。

5.4 正则化

我们在训练过程中采用了三种类型的正则化：

残差丢弃我们对每个子层的输出应用丢弃 [27]，在其被添加到子层输入并归一化之前。此外，我们对编码器和解码器堆叠中嵌入与位置编码的和也应用丢弃。对于基础模型，我们使用的丢弃率为 $P_{drop} = 0.1$ 。

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.0	$2.3 \cdot 10^{19}$	

Label Smoothing During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [30]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [31]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [31].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU ⁵.

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

⁵We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

表 2: 在英语到德语和英语到法语的 newstest2014 测试中, Transformer 以极低的训练成本达到了比之前最先进模型更好的 BLEU 分数。

模型	BLEU		训练成本 (FLOPs)	
	英语到德语	英语到法语	英语到德 语	英语到法 语
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT +RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att +PosUnk 集成 [32]		40.4		$8.0 \cdot 10^{20}$
GNMT +RL 集成 [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [8]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (基础模型)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (大模型)	28.4	41.0	$2.3 \cdot 10^{19}$	

标签平滑在训练期间, 我们采用了值为 $\epsilon_{ls} = 0.1$ [30]的标签平滑。这会损害困惑度, 因为模型学会了更加不确定, 但提高了准确率和BLEU 分数。

6 结果

6.1 机器翻译

在 WMT 2014 英语到德语翻译任务上, 大型 Transformer 模型 (表2中的 Transformer (big)) 以超过 2.0 BLEU 的成绩超越了此前报告的最佳模型 (包括集成模型), 创下了 28.4 的最先进 BLEU 分数。该模型的配置列于表3的最后一行。训练在 8 块 P100 GPU 上耗时 3.5 天。即便我们的基础模型也超越了此前发表的所有模型和集成模型, 而训练成本仅为任何竞争模型的一小部分。

在 WMT 2014 英语到法语翻译任务上, 我们的大型模型取得了 41.0 的 BLEU 分数, 超越了此前发表的所有单模型, 且训练成本不到先前最先进模型的 1/4。用于英语到法语翻译训练的 Transformer (big) 模型使用了 dropout 率 $P_{drop} = 0.1$, 而非 0.3。

对于基础模型, 我们使用了通过对最后5个检查点取平均得到的单一模型, 这些检查点以 10分钟为间隔保存。对于大型模型, 我们对最后20个检查点取平均。我们使用了束宽为4、长度惩罚为 $\alpha = 0.6$ [31]的束搜索。这些超参数是在开发集上经过实验后选定的。我们在推理时将最大输出长度设为输入长度 + 50, 但在可能时提前终止 [31]。

表2总结了我们的结果, 并将我们的翻译质量和训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、使用的GPU数量以及每个GPU的持续单精度浮点容量估计值相乘来估算训练模型所用的浮点运算次数 ⁵。

6.2 模型变体

为了评估Transformer不同组件的重要性, 我们以不同方式对基础模型进行了变体, 并测量了在开发集newstest2013上英语到德语翻译性能的变化。我们使用了上一节所述的束搜索, 但未使用检查点平均。我们在表3中展示了这些结果。

在表3的行(A)中, 我们改变注意力头的数量以及注意力键和值的维度, 同时保持计算量不变, 如第3.2.2节所述。虽然单头注意力比最佳设置差0.9 BLEU, 但注意力头过多时质量也会下降。

⁵我们使用了 2.8、3.7、6.0 和 9.5 TFLOPS 的数值, 分别对应 K80、K40、M40 和 P100。

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$	
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65	
(A)					1	512	512				5.29	24.9	
					4	128	128				5.00	25.5	
					16	32	32				4.91	25.8	
					32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58	
					32					5.01	25.4	60	
(C)	2									6.11	23.7	36	
	4									5.19	25.3	50	
	8									4.88	25.5	80	
		256				32	32				5.75	24.5	28
		1024				128	128				4.66	26.0	168
			1024								5.12	25.4	53
			4096								4.75	26.2	90
(D)							0.0			5.77	24.6		
							0.2			4.95	25.5		
								0.0		4.67	25.3		
								0.2		5.47	25.7		
(E)	positional embedding instead of sinusoids									4.92	25.7		
big	6	1024	4096	16					0.3	300K	4.33	26.4	213

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [8], and observe nearly identical results to the base model.

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

The code we used to train and evaluate our models is available at <https://github.com/tensorflow/tensor2tensor>.

Acknowledgements We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

表3: Transformer架构的变体。未列出的值与基础模型相同。所有指标均基于英德翻译开发集newstest2013。所列困惑度为按子词计算的困惑度，依据我们的字节对编码，不应与按词困惑度进行比较。

	N	$d_{\text{模型}}$	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	训练 步数	PPL (开发集)	BLEU (开发集)	参数量 $\times 10^6$				
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65				
(A)										5.29	24.9					
										5.00	25.5					
										4.91	25.8					
										5.01	25.4					
(B)										5.16	25.1	58				
										5.01	25.4	60				
(C)	2										6.11	23.7	36			
	4										256	32	32	5.19	25.3	50
	8													4.88	25.5	80
	1024													5.75	24.5	28
											1024	4.66	26.0	168		
												4096	5.12	25.4	53	
	4.75												26.2	90		
(D)										5.77	24.6					
										4.95	25.5					
										4.67	25.3					
										5.47	25.7					
(E)	位置嵌入代替正弦波									4.92	25.7					
big	6	1024	4096	16	0.3				300K	4.33	26.4	213				

在表3的行(B)中，我们观察到减小注意力键大小 d_k 会损害模型质量。这表明确定兼容性并非易事，并且比点积更复杂的兼容性函数可能是有益的。我们在行(C)和(D)中进一步观察到，正如预期的那样，更大的模型更好，而丢弃在避免过拟合方面非常有帮助。在行(E)中，我们将正弦位置编码替换为学习的位置嵌入 [8]，并观察到与基础模型几乎相同的结果。

7 结论

在这项工作中，我们提出了Transformer，这是第一个完全基于注意力的序列转换模型，用**多头自注意力**取代了编码器-解码器架构中最常用的循环层。

对于翻译任务，Transformer的训练速度显著快于基于循环层或卷积层的架构。在WMT 2014英德和WMT 2014英法翻译任务上，我们都达到了新的最先进水平。在前一项任务中，我们最好的模型甚至超越了此前报告的所有集成模型。

我们对基于注意力的模型的将来感到兴奋，并计划将其应用于其他任务。我们计划将Transformer扩展到涉及文本以外的输入输出模态的问题，并研究局部受限注意力机制，以高效处理图像、音频和视频等大型输入和输出。使生成过程不那么序列化也是我们的另一个研究目标。

我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们感谢Nal Kalchbrenner和Stephan Gouws提出的富有成果的评论、修正和启发。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

[5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.

[8] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.

[9] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[11] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[12] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[13] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[14] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[15] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[16] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[17] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[18] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[19] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[20] Samy Bengio Łukasz Kaiser. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

参考文献

[1] Jimmy Lei Ba、Jamie Ryan Kiros 和 Geoffrey E Hinton。层归一化。*arXiv* 预印本 *arXiv:1607.06450*，2016。[2] Dzmitry Bahdanau、Kyunghyun Cho 和 Yoshua Bengio。通过联合学习对齐和翻译实现神经机器翻译。*CoRR*，abs/1409.0473，2014。[3] Denny Britz、Anna Goldie、Minh-Thang Luong 和 Quoc V. Le。神经机器翻译架构的大规模探索。*CoRR*，abs/1703.03906，2017。[4] Jianpeng Cheng、Li Dong 和 Mirella Lapata。用于机器阅读的长短期记忆网络。*arXiv* 预印本 *arXiv:1601.06733*，2016。[5] Kyunghyun Cho、Bart van Merriënboer、Çağlar Gulcehre、Fethi Bougares、Holger Schwenk 和 Yoshua Bengio。使用 RNN 编码器-解码器学习短语表征用于统计机器翻译。*CoRR*，abs/1406.1078，2014。[6] Francois Chollet。Xception：使用深度可分离卷积的深度学习。*arXiv* 预印本 *arXiv:1610.02357*，2016。[7] Junyoung Chung、Çağlar Gülçehre、Kyunghyun Cho 和 Yoshua Bengio。门控循环神经网络在序列建模上的实证评估。*CoRR*，abs/1412.3555，2014。[8] Jonas Gehring、Michael Auli、David Grangier、Denis Yarats 和 Yann N. Dauphin。卷积序列到序列学习。*arXiv* 预印本 *arXiv:1705.03122v2*，2017。[9] Alex Graves。使用循环神经网络生成序列。*arXiv* 预印本 *arXiv:1308.0850*，2013。[10] Kaiming He、Xiangyu Zhang、Shaoqing Ren 和 Jian Sun。用于图像识别的深度残差学习。发表于 *IEEE 计算机视觉与模式识别会议论文集*，第 770–778 页，2016。[11] Sepp Hochreiter、Yoshua Bengio、Paolo Frasconi 和 Jürgen Schmidhuber。循环网络中的梯度流：学习长期依赖关系的困难，2001。[12] Sepp Hochreiter 和 Jürgen Schmidhuber。长短期记忆。神经计算，9(8):1735–1780，1997。[13] Rafal Jozefowicz、Oriol Vinyals、Mike Schuster、Noam Shazeer 和 Yonghui Wu。探索语言建模的极限。*arXiv* 预印本 *arXiv:1602.02410*，2016。[14] Łukasz Kaiser 和 Ilya Sutskever。神经GPU 学习算法。发表于国际学习表征会议（*ICLR*），2016。[15] Nal Kalchbrenner、Lasse Espeholt、Karen Simonyan、Aaron van den Oord、Alex Graves 和 Koray Kavukcuoglu。线性时间中的神经机器翻译。*arXiv* 预印本 *arXiv:1610.10099v2*，2017。[16] Yoon Kim、Carl Denton、Luong Hoang 和 Alexander M. Rush。结构化注意力网络。发表于国际学习表征会议，2017。[17] Diederik Kingma 和 Jimmy Ba。Adam：一种随机优化方法。发表于 *ICLR*，2015。[18] Oleksii Kuchaiev 和 Boris Ginsburg。LSTM 网络的分解技巧。*arXiv* 预印本 *arXiv:1703.10722*，2017。[19] Zhouhan Lin、Minwei Feng、Cicero Nogueira dos Santos、Mo Yu、Bing Xiang、Bowen Zhou 和 Yoshua Bengio。一种结构化自注意力句子嵌入。*arXiv* 预印本 *arXiv:1703.03130*，2017。[20] Samy Bengio Łukasz Kaiser。主动记忆能否取代注意力？发表于神经信息处理系统进展（*NIPS*），2016。

- [21] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
- [22] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [23] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [24] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [25] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [26] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [27] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [28] Sainbayar Sukhbaatar, arthur szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [29] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [30] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [31] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [32] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[21] Minh-Thang Luong、Hieu Pham 和 Christopher D Manning。基于注意力的神经机器翻译的有效方法。*arXiv*预印本 *arXiv:1508.04025*，2015年。[22] Ankur Parikh、Oscar Täckström、Dipanjan Das 和 Jakob Uszkoreit。可分解注意力模型。载于自然语言处理经验方法，2016年。[23] Romain Paulus、Caiming Xiong 和 Richard Socher。用于抽象摘要的深度强化模型。*arXiv*预印本 *arXiv:1705.04304*，2017年。[24] Ofir Press 和 Lior Wolf。利用输出嵌入改进语言模型。*arXiv*预印本 *arXiv:1608.05859*，2016年。[25] Rico Sennrich、Barry Haddow 和 Alexandra Birch。使用子词单元的稀有词神经机器翻译。*arXiv*预印本 *arXiv:1508.07909*，2015年。[26] Noam Shazeer、Azalia Mirhoseini、Krzysztof Maziarczyk、Andy Davis、Quoc Le、Geoffrey Hinton 和 Jeff Dean。超大型神经网络：稀疏门控专家混合层。*arXiv*预印本 *arXiv:1701.06538*，2017年。[27] Nitish Srivastava、Geoffrey E Hinton、Alex Krizhevsky、Ilya Sutskever 和 Ruslan Salakhutdinov。丢弃：一种防止神经网络过拟合的简单方法。机器学习研究杂志，15(1):1929–1958，2014年。[28] Sainbayar Sukhbaatar、arthur szlam、Jason Weston 和 Rob Fergus。端到端记忆网络。载于 C. Cortes、N. D. Lawrence、D. D. Lee、M. Sugiyama 和 R. Garnett 编，神经信息处理系统进展 28，第 2440–2448页。Curran Associates, Inc.，2015年。[29] Ilya Sutskever、Oriol Vinyals 和 Quoc VV Le。基于神经网络的序列到序列学习。载于神经信息处理系统进展，第3104–3112页，2014年。[30] Christian Szegedy、Vincent Vanhoucke、Sergey Ioffe、Jonathon Shlens 和 Zbigniew Wojna。重新思考计算机视觉的Inception架构。*CoRR*，abs/1512.00567，2015年。[31] Yonghui Wu、Mike Schuster、Zhifeng Chen、Quoc V Le、Mohammad Norouzi、Wolfgang Macherey、Maxim Krikun、Yuan Cao、Qin Gao、Klaus Macherey 等。Google的神经机器翻译系统：弥合人类和机器翻译之间的差距。*arXiv*预印本 *arXiv:1609.08144*，2016年。[32] Jie Zhou、Ying Cao、Xuguang Wang、Peng Li 和 Wei Xu。带有快速前向连接的深度循环模型用于神经机器翻译。*CoRR*，abs/1606.04199，2016年。